

УО «Белорусский государственный университет информатики и
радиоэлектроники»
Кафедра ПОИТ

Отчет по лабораторной работе №4
по предмету
Распределенные вычисления
Вариант 3

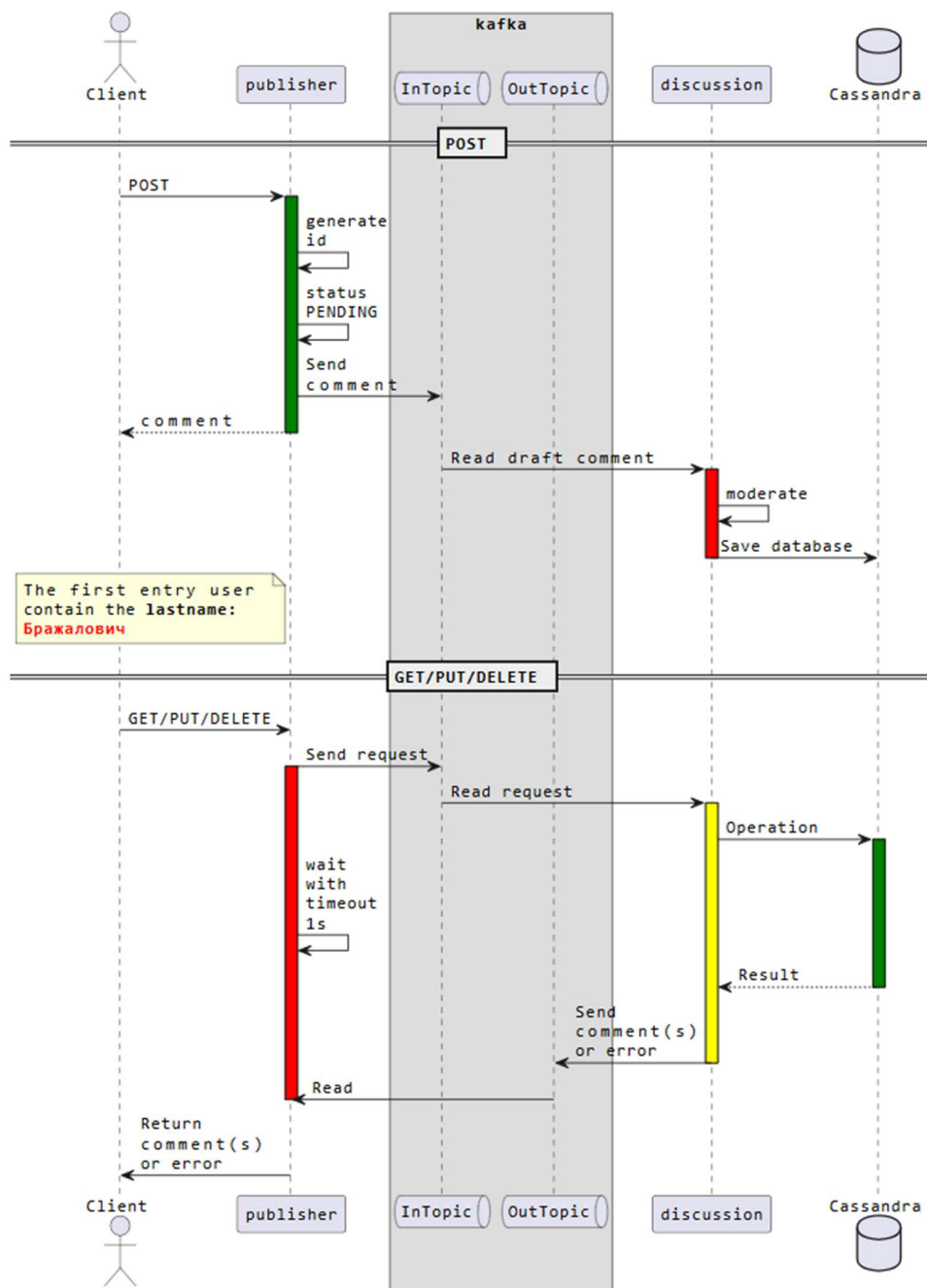
Выполнил:
Бражалович А.И.

Проверил:
Данилова Г.В.

Группа 351004

Минск 2026

Задание



Код

```

package org.example.newsapi.exception;

public class AlreadyExistsException extends RuntimeException {
    public AlreadyExistsException(String message) {
        super(message);
    }
}
  
```

```
}  
}
```

=====
File: Files from 4/application_1.properties
=====

???????????? ?????????? (PostgreSQL, ??? ?????)

server.port=24110

spring.datasource.url=jdbc:postgresql://localhost:5432/distcomp

spring.datasource.username=postgres

spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=validate

spring.liquibase.change-log=classpath:/db/changelog/db.changelog-master.xml

--- ?????????? Kafka (?????????) ---

spring.kafka.bootstrap-servers=localhost:9092

spring.kafka.consumer.group-id=publisher-group

spring.kafka.consumer.auto-offset-reset=earliest

spring.kafka.consumer.key-

deserializer=org.apache.kafka.common.serialization.StringDeserializer

spring.kafka.consumer.value-

deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.trusted.packages=*

spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer

spring.kafka.producer.value-

serializer=org.springframework.kafka.support.serializer.JsonSerializer

????? ??????? (???????????? ? ????)

app.kafka.in-topic=in-topic

app.kafka.out-topic=out-topic

=====
File: Files from 4/application.properties
=====

???????????? ?????????? (Cassandra, ??? ?????)

spring.cassandra.contact-points=localhost

server.port=24110

spring.cassandra.port=9042

spring.cassandra.local-datacenter=datacenter1

spring.cassandra.keyspace-name=distcomp

spring.cassandra.schema-action=CREATE_IF_NOT_EXISTS

--- ?????????? Kafka (?????????) ---

spring.kafka.bootstrap-servers=localhost:9092

spring.kafka.consumer.group-id=discussion-group

```
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-
deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages=*
```

```
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-
serializer=org.springframework.kafka.support.serializer.JsonSerializer
```

```
# ????? ?????? (???????????? ? ????)
app.kafka.in-topic=in-topic
app.kafka.out-topic=out-topic
```

```
=====
File: Files from 4/CassandraConfig.java
=====
```

```
package org.example.discussion.config;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.cassandra.config.AbstractCassandraConfiguration;
import org.springframework.data.cassandra.config.SchemaAction;
import org.springframework.data.cassandra.core.cql.keyspace.CreateKeyspaceSpecification;
import org.springframework.data.cassandra.core.cql.keyspace.KeyspaceOption;
import java.util.List;
@Configuration
public class CassandraConfig extends AbstractCassandraConfiguration {
    @Override
    protected String getKeyspaceName() {
        return "distcomp";
    }
    @Override
    protected String getContactPoints() {
        return "localhost";
    }
    @Override
    protected int getPort() {
        return 9042;
    }
    @Override
    public SchemaAction getSchemaAction() {
        return SchemaAction.CREATE_IF_NOT_EXISTS;
    }
    @Override
    protected List<CreateKeyspaceSpecification> getKeyspaceCreations() {
        return List.of(CreateKeyspaceSpecification.createKeyspace(getKeyspaceName()))
    }
}
```

```

        .ifNotExists()
        .with(KeyspaceOption.DURABLE_WRITES, true)
        .withSimpleReplication());
    }
    @Override
    public String[] getEntityBasePackages() {
        return new String[]{"org.example.discussion.entity"};
    }
}

```

=====

File: Files from 4/Comment.java

=====

```

package org.example.discussion.entity;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;
@Table("tbl_comment")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Comment {
    @PrimaryKey
    private Long id;
    @Column("news_id")
    private Long newsId;
    private String content;
    private String state;
}

```

=====

File: Files from 4/CommentController_1.java

=====

```

package org.example.newsapi.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.example.newsapi.service.CommentService;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;

```

```

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/v1.0/comments")
@RequiredArgsConstructor
public class CommentController {
    private final CommentService commentService;
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public CommentResponseTo create(@RequestBody @Valid CommentRequestTo request) {
        return commentService.create(request);
    }
    @GetMapping
    public List<CommentResponseTo> getAll(@PageableDefault(size = 50) Pageable pageable) {
        return commentService.findAll();
    }
    @GetMapping("/{id}")
    public CommentResponseTo getById(@PathVariable Long id) {
        return commentService.findById(id);
    }
    @PutMapping("/{id}")
    public CommentResponseTo update(@PathVariable Long id, @RequestBody @Valid
CommentRequestTo request) {
        return commentService.update(id, request);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        commentService.delete(id);
    }
}

```

=====

File: Files from 4/CommentController.java

=====

```

package org.example.discussion.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.discussion.dto.request.CommentRequestTo;
import org.example.discussion.dto.response.CommentResponseTo;
import org.example.discussion.service.CommentService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController

```

```

@RequestMapping("/api/v1.0/comments")
@RequiredArgsConstructor
public class CommentController {
    private final CommentService commentService;
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public CommentResponseTo create(@RequestBody @Valid CommentRequestTo request) {
        return commentService.create(request);
    }
    @GetMapping
    public List<CommentResponseTo> getAll() {
        return commentService.findAll();
    }
    @GetMapping("/{id}")
    public CommentResponseTo getById(@PathVariable Long id) {
        return commentService.findById(id);
    }
    @PutMapping("/{id}")
    public CommentResponseTo update(@PathVariable Long id, @RequestBody @Valid
CommentRequestTo request) {
        return commentService.update(id, request);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        commentService.delete(id);
    }
}

```

=====
File: Files from 4/CommentKafkaMessage_1.java
=====

```

package org.example.newsapi.dto.kafka;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class CommentKafkaMessage {
    private Long id;
    private Long newsId;
    private String content;
    private String state;
}

```

=====

File: Files from 4/CommentKafkaMessage.java

=====

```
package org.example.discussion.dto.kafka;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class CommentKafkaMessage {
    private Long id;
    private Long newsId;
    private String content;
    private String state;
}
```

=====

File: Files from 4/CommentMapper.java

=====

```
package org.example.discussion.mapper;
import org.example.discussion.dto.request.CommentRequestTo;
import org.example.discussion.dto.response.CommentResponseTo;
import org.example.discussion.entity.Comment;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;
@Mapper(componentModel = "spring")
public interface CommentMapper {
    @Mapping(target = "id", ignore = true)
    Comment toEntity(CommentRequestTo request);
    CommentResponseTo toDto(Comment comment);
    @Mapping(target = "id", ignore = true)
    void updateEntityFromDto(CommentRequestTo request, @MappingTarget Comment
comment);
}
```

=====

File: Files from 4/CommentRepository.java

=====

```
package org.example.discussion.repository;
import org.example.discussion.entity.Comment;
import org.springframework.data.cassandra.repository.CassandraRepository;
import org.springframework.data.cassandra.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;
```

```
import java.util.Optional;
@Repository
public interface CommentRepository extends CassandraRepository<Comment, Long> {
    @Query("SELECT * FROM tbl_comment WHERE news_id = ?0 ALLOW FILTERING")
    List<Comment> findByNewsId(Long newsId);
    Optional<Comment> findById(Long id);
}
```

=====

File: Files from 4/CommentRequestTo_1.java

=====

```
package org.example.newsapi.dto.request;
import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;
@Data
public class CommentRequestTo {
    @NotNull
    private Long newsId;
    @Size(min = 2, max = 2048)
    private String content;
    public void setNews(Long news) {
        this.newsId = news;
    }
}
```

=====

File: Files from 4/CommentRequestTo.java

=====

```
package org.example.discussion.dto.request;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class CommentRequestTo {
    @NotNull
    private Long newsId;
    @Size(min = 2, max = 2048)
    private String content;
}
```

=====

File: Files from 4/CommentResponseTo_1.java

=====

```
package org.example.newsapi.dto.response;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class CommentResponseTo {
    private Long id;
    private Long newsId;
    private String content;
    public Long getNews() {
        return this.newsId;
    }
}
```

=====

File: Files from 4/CommentResponseTo.java

=====

```
package org.example.discussion.dto.response;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class CommentResponseTo {
    private Long id;
    private Long newsId;
    private String content;
}
```

=====

File: Files from 4/CommentService_1.java

=====

```
package org.example.newsapi.service;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.example.newsapi.exception.NotFoundException;
import org.springframework.stereotype.Service;
import java.util.List;
```

```

@Service
@RequiredArgsConstructor
public class CommentService {
    private final KafkaCommentService kafkaCommentService;
    public CommentResponseTo create(CommentRequestTo request) {
        return kafkaCommentService.createComment(request);
    }
    public List<CommentResponseTo> findAll() {
        throw new UnsupportedOperationException("Not implemented yet");
    }
    public CommentResponseTo findById(Long id) {
        throw new UnsupportedOperationException("Not implemented yet");
    }
    public CommentResponseTo update(Long id, CommentRequestTo request) {
        throw new UnsupportedOperationException("Not implemented yet");
    }
    public void delete(Long id) {
        throw new UnsupportedOperationException("Not implemented yet");
    }
}

```

=====

File: Files from 4/CommentService.java

=====

```

package org.example.discussion.service;
import lombok.RequiredArgsConstructor;
import org.example.discussion.dto.request.CommentRequestTo;
import org.example.discussion.dto.response.CommentResponseTo;
import org.example.discussion.entity.Comment;
import org.example.discussion.exception.NotFoundException;
import org.example.discussion.mapper.CommentMapper;
import org.example.discussion.repository.CommentRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.stream.Collectors;
@Service
@RequiredArgsConstructor
public class CommentService {
    private final CommentRepository commentRepository;
    private final CommentMapper commentMapper;
    private final IdGenerator idGenerator = new IdGenerator();
    public CommentResponseTo create(CommentRequestTo request) {
        Comment comment = commentMapper.toEntity(request);
        comment.setId(idGenerator.nextId());
        comment.setState("APPROVE");
        Comment saved = commentRepository.save(comment);
    }
}

```

```

        return commentMapper.toDto(saved);
    }
    public List<CommentResponseTo> findAll() {
        return commentRepository.findAll()
            .stream()
            .map(commentMapper::toDto)
            .collect(Collectors.toList());
    }
    public CommentResponseTo findById(Long id) {
        Comment comment = commentRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("Comment not found with id: " + id));
        return commentMapper.toDto(comment);
    }
    public CommentResponseTo update(Long id, CommentRequestTo request) {
        Comment comment = commentRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("Comment not found with id: " + id));
        commentMapper.updateEntityFromDto(request, comment);
        comment.setState("APPROVE");
        Comment saved = commentRepository.save(comment);
        return commentMapper.toDto(saved);
    }
    public void delete(Long id) {
        if (!commentRepository.existsById(id)) {
            throw new NotFoundException("Comment not found with id: " + id);
        }
        commentRepository.deleteById(id);
    }
}

```

=====
File: Files from 4/db.changelog-master.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
    xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.24.xsd">
    <include file="changeset/v1.0.0-create-tables.xml" relativeToChangelogFile="true"/>
</databaseChangeLog>

```

=====
File: Files from 4/DiscussionApplication.java

```

package org.example.discussion;
import org.springframework.boot.CommandLineRunner;

```

```

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.cassandra.core.CassandraTemplate;
import org.springframework.util.FileCopyUtils;
import java.nio.charset.StandardCharsets;
@SpringBootApplication
public class DiscussionApplication implements CommandLineRunner {
    private final CassandraTemplate cassandraTemplate;
    public DiscussionApplication(CassandraTemplate cassandraTemplate) {
        this.cassandraTemplate = cassandraTemplate;
    }
    public static void main(String[] args) {
        SpringApplication.run(DiscussionApplication.class, args);
    }
    @Override
    public void run(String... args) throws Exception {
        ClassPathResource resource = new ClassPathResource("schema.cql");
        byte[] bdata = FileCopyUtils.copyToByteArray(resource.getInputStream());
        String cql = new String(bdata, StandardCharsets.UTF_8);
        String[] statements = cql.split(";");
        for (String stmt : statements) {
            if (!stmt.trim().isEmpty()) {
                cassandraTemplate.getCqlOperations().execute(stmt);
            }
        }
    }
}

```

=====
File: Files from 4/ErrorResponse_1.java
=====

```

package org.example.newsapi.exception;
import lombok.AllArgsConstructor;
import lombok.Data;
@Data
@AllArgsConstructor
public class ErrorResponse {
    private String errorMessage;
    private int errorCode;
}

```

=====
File: Files from 4/ErrorResponse.java
=====

```

package org.example.discussion.exception;

```

```

import lombok.Data;
@Data
public class ErrorResponse {
    private String errorMessage;
    private int errorCode;
    public ErrorResponse(String errorMessage, int errorCode) {
        this.errorMessage = errorMessage;
        this.errorCode = errorCode;
    }
}

```

=====
File: Files from 4/GlobalExceptionHandler_1.java
=====

```

package org.example.newsapi.exception;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler({NotFoundException.class, AlreadyExistsException.class})
    public ResponseEntity<ErrorResponse> handleBusinessError(RuntimeException e) {
        System.out.println(">>> HANDLED BUSINESS ERROR: " + e.getMessage());
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ErrorResponse(e.getMessage(), 40301));
    }
    @ExceptionHandler(org.springframework.dao.DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleSqlError(Exception e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ErrorResponse("Database error", 40301));
    }
    @ExceptionHandler(org.springframework.web.bind.MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationError(Exception e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ErrorResponse("Validation error", 40301));
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse(e.getMessage(), 50000));
    }
}

```

=====

File: Files from 4/GlobalExceptionHandler.java

```
=====
package org.example.discussion.exception;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(NotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(e.getMessage(), 40401));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(MethodArgumentNotValidException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("Validation error", 40001));
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse(e.getMessage(), 50000));
    }
}
=====
```

File: Files from 4/IdGenerator_1.java

```
=====
package org.example.newsapi.service;
import org.springframework.stereotype.Component;
@Component
public class IdGenerator {
    private static final long EPOCH = 1735689600000L;
    private static final int NODE_ID_BITS = 10;
    private static final int SEQUENCE_BITS = 12;
    private static final long MAX_SEQUENCE = ~(-1L << SEQUENCE_BITS);
    private static final long MAX_NODE_ID = ~(-1L << NODE_ID_BITS);
    private final long nodeId;
    private long lastTimestamp = -1L;
    private long sequence = 0L;
    public IdGenerator() {
        this.nodeId = 1L;
        if (nodeId < 0 || nodeId > MAX_NODE_ID) {
```

```

        throw new IllegalArgumentException(String.format("NodeId must be between 0 and
%d", MAX_NODE_ID));
    }
}

public synchronized long nextId() {
    long timestamp = System.currentTimeMillis();
    if (timestamp < lastTimestamp) {
        throw new RuntimeException("Clock moved backwards");
    }
    if (timestamp == lastTimestamp) {
        sequence = (sequence + 1) & MAX_SEQUENCE;
        if (sequence == 0) {
            timestamp = waitNextMillis(timestamp);
        }
    } else {
        sequence = 0L;
    }
    lastTimestamp = timestamp;
    return ((timestamp - EPOCH) << (NODE_ID_BITS + SEQUENCE_BITS))
        | (nodeId << SEQUENCE_BITS)
        | sequence;
}

private long waitNextMillis(long currentTimestamp) {
    while (currentTimestamp == lastTimestamp) {
        currentTimestamp = System.currentTimeMillis();
    }
    return currentTimestamp;
}
}

```

=====
File: Files from 4/IdGenerator.java
=====

```

package org.example.discussion.service;
import org.springframework.stereotype.Component;
@Component
public class IdGenerator {
    private static final long EPOCH = 1735689600000L;
    private static final int NODE_ID_BITS = 10;
    private static final int SEQUENCE_BITS = 12;
    private static final long MAX_SEQUENCE = ~(-1L << SEQUENCE_BITS);
    private static final long MAX_NODE_ID = ~(-1L << NODE_ID_BITS);
    private final long nodeId;
    private long lastTimestamp = -1L;
    private long sequence = 0L;
    public IdGenerator() {

```

```

        this.nodeId = 1L;
        if (nodeId < 0 || nodeId > MAX_NODE_ID) {
            throw new IllegalArgumentException(String.format("NodeId must be between 0 and
%d", MAX_NODE_ID));
        }
    }
    public synchronized long nextId() {
        long timestamp = System.currentTimeMillis();
        if (timestamp < lastTimestamp) {
            throw new RuntimeException("Clock moved backwards");
        }
        if (timestamp == lastTimestamp) {
            sequence = (sequence + 1) & MAX_SEQUENCE;
            if (sequence == 0) {
                timestamp = waitNextMillis(timestamp);
            }
        } else {
            sequence = 0L;
        }
        lastTimestamp = timestamp;
        return ((timestamp - EPOCH) << (NODE_ID_BITS + SEQUENCE_BITS))
            | (nodeId << SEQUENCE_BITS)
            | sequence;
    }
    private long waitNextMillis(long currentTimestamp) {
        while (currentTimestamp == lastTimestamp) {
            currentTimestamp = System.currentTimeMillis();
        }
        return currentTimestamp;
    }
}

```

=====

File: Files from 4/KafkaCommentConsumer.java

=====

```

package org.example.discussion.service;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.discussion.dto.kafka.CommentKafkaMessage;
import org.example.discussion.entity.Comment;
import org.example.discussion.repository.CommentRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import java.util.Arrays;

```

```

import java.util.HashSet;
import java.util.Set;
@Service
@RequiredArgsConstructor
@Slf4j
public class KafkaCommentConsumer {
    private final CommentRepository commentRepository;
    private final KafkaTemplate<String, CommentKafkaMessage> kafkaTemplate;
    @Value("${app.kafka.out-topic}")
    private String outTopic;
    private static final Set<String> STOP_WORDS = new HashSet<>(Arrays.asList("spam",
"bad", "offensive"));
    @KafkaListener(topics = "${app.kafka.in-topic}", groupId =
"${spring.kafka.consumer.group-id}")
    public void consume(CommentKafkaMessage message) {
        log.info("Received message from InTopic: {}", message);
        String state = moderate(message.getContent()) ? "APPROVE" : "DECLINE";
        if ("APPROVE".equals(state)) {
            Comment comment = Comment.builder()
                .id(message.getId())
                .newsId(message.getNewsId())
                .content(message.getContent())
                .build();
            commentRepository.save(comment);
        }
        CommentKafkaMessage response = new CommentKafkaMessage(
            message.getId(),
            message.getNewsId(),
            message.getContent(),
            state
        );
        kafkaTemplate.send(outTopic, String.valueOf(message.getNewsId()), response);
        log.info("Sent response to OutTopic: {}", response);
    }
    private boolean moderate(String content) {
        for (String word : STOP_WORDS) {
            if (content.toLowerCase().contains(word.toLowerCase())) {
                return false;
            }
        }
        return true;
    }
}

```

File: Files from 4/KafkaCommentService.java

```

=====
package org.example.newsapi.service;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.newsapi.dto.kafka.CommentKafkaMessage;
import org.example.newsapi.dto.request.CommentRequestTo;
import org.example.newsapi.dto.response.CommentResponseTo;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.TimeUnit;
@Service
@RequiredArgsConstructor
@Slf4j
public class KafkaCommentService {
    private final KafkaTemplate<String, CommentKafkaMessage> kafkaTemplate;
    private final IdGenerator idGenerator;
    @Value("${app.kafka.in-topic}")
    private String inTopic;
    @Value("${app.kafka.out-topic}")
    private String outTopic;
    private final ConcurrentHashMap<Long, CompletableFuture<CommentKafkaMessage>>
pendingRequests = new ConcurrentHashMap<>();
    public CommentResponseTo createComment(CommentRequestTo request) {
        Long id = idGenerator.nextId();
        CommentKafkaMessage message = new CommentKafkaMessage(id, request.getNewsId(),
request.getContent(), "PENDING");
        CompletableFuture<CommentKafkaMessage> future = new CompletableFuture<>();
        pendingRequests.put(id, future);
        kafkaTemplate.send(inTopic, String.valueOf(request.getNewsId()), message);
        try {
            CommentKafkaMessage response = future.get(1, TimeUnit.SECONDS);
            if ("APPROVE".equals(response.getState())) {
                return new CommentResponseTo(response.getId(), response.getNewsId(),
response.getContent());
            } else {
                throw new RuntimeException("Comment declined by moderator");
            }
        } catch (Exception e) {
            log.error("Error while waiting for Kafka response", e);
            throw new RuntimeException("Comment processing timeout or error");
        } finally {
            pendingRequests.remove(id);

```

```

    }
}
@KafkaListener(topics = "${app.kafka.out-topic}", groupId =
"${spring.kafka.consumer.group-id}")
public void listen(CommentKafkaMessage message) {
    log.info("Received response from OutTopic: {}", message);
    CompletableFuture<CommentKafkaMessage> future =
pendingRequests.get(message.getId());
    if (future != null) {
        future.complete(message);
    } else {
        log.warn("No pending request for comment id: {}", message.getId());
    }
}
}
}

```

=====

File: Files from 4/Marker.java

=====

```

package org.example.newsapi.entity;
import jakarta.persistence.*;
import lombok.*;
@Entity
@Table(name = "tbl_marker")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Marker {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "marker_seq_gen")
    @SequenceGenerator(name = "marker_seq_gen", sequenceName = "marker_seq",
allocationSize = 1)
    private Long id;
    @Column(unique = true, nullable = false, length = 32)
    private String name;
}

```

=====

File: Files from 4/MarkerController.java

=====

```

package org.example.newsapi.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;

```

```

import org.example.newsapi.service.MarkerService;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/v1.0/markers")
@RequiredArgsConstructor
public class MarkerController {
    private final MarkerService markerService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public MarkerResponseTo create(@RequestBody @Valid MarkerRequestTo request) {
        return markerService.create(request);
    }

    @GetMapping
    public List<MarkerResponseTo> getAll(@PageableDefault(size = 50) Pageable pageable) {
        return markerService.findAll(pageable).getContent();
    }

    @GetMapping("/{id}")
    public MarkerResponseTo getById(@PathVariable Long id) {
        return markerService.findById(id);
    }

    @PutMapping("/{id}")
    public MarkerResponseTo update(@PathVariable Long id, @RequestBody @Valid
MarkerRequestTo request) {
        return markerService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        markerService.delete(id);
    }
}

```

=====
File: Files from 4/MarkerMapper.java
=====

```

package org.example.newsapi.mapper;
import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;
import org.example.newsapi.entity.Marker;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

```

```

@Mapper(componentModel = "spring")
public interface MarkerMapper {
    @Mapping(target = "id", ignore = true)
    Marker toEntity(MarkerRequestTo request);
    MarkerResponseTo toDto(Marker marker);
    @Mapping(target = "id", ignore = true)
    void updateEntityFromDto(MarkerRequestTo request, @MappingTarget Marker marker);
}

```

=====

File: Files from 4/MarkerRepository.java

=====

```

package org.example.newsapi.repository;
import org.example.newsapi.entity.Marker;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
@Repository
public interface MarkerRepository extends JpaRepository<Marker, Long> {
    boolean existsByName(String name);
    Optional<Marker> findByName(String name);
}

```

=====

File: Files from 4/MarkerRequestTo.java

=====

```

package org.example.newsapi.dto.request;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class MarkerRequestTo {
    @NotBlank
    @Size(min = 2, max = 32)
    private String name;
}

```

=====

File: Files from 4/MarkerResponseTo.java

=====

```

package org.example.newsapi.dto.response;
import lombok.Data;

```

```

@Data
public class MarkerResponseTo {
    private Long id;
    private String name;
}

```

=====
File: Files from 4/MarkerService.java
=====

```

package org.example.newsapi.service;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.MarkerRequestTo;
import org.example.newsapi.dto.response.MarkerResponseTo;
import org.example.newsapi.entity.Marker;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.MarkerMapper;
import org.example.newsapi.repository.MarkerRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class MarkerService {
    private final MarkerRepository markerRepository;
    private final MarkerMapper markerMapper;
    @Transactional
    public MarkerResponseTo create(MarkerRequestTo request) {
        if (markerRepository.existsByName(request.getName())) {
            throw new AlreadyExistsException("Marker already exists");
        }
        Marker marker = markerMapper.toEntity(request);
        Marker saved = markerRepository.save(marker);
        return markerMapper.toDto(saved);
    }
    public Page<MarkerResponseTo> findAll(Pageable pageable) {
        return markerRepository.findAll(pageable)
            .map(markerMapper::toDto);
    }
    public MarkerResponseTo findById(Long id) {
        return markerRepository.findById(id)
            .map(markerMapper::toDto)
            .orElseThrow(() -> new NotFoundException("Marker not found with id: " + id));
    }
}

```

```

@Transactional
public MarkerResponseTo update(Long id, MarkerRequestTo request) {
    Marker marker = markerRepository.findById(id)
        .orElseThrow(() -> new NotFoundException("Marker not found with id: " + id));
    markerMapper.updateEntityFromDto(request, marker);
    return markerMapper.toDto(markerRepository.save(marker));
}
@Transactional
public void delete(Long id) {
    if (!markerRepository.existsById(id)) {
        throw new NotFoundException("Marker not found with id: " + id);
    }
    markerRepository.deleteById(id);
}
}

```

=====

File: Files from 4/News.java

=====

```

package org.example.newsapi.entity;
import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;
@Entity
@Table(name = "tbl_news")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class News {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "news_seq_gen")
    @SequenceGenerator(name = "news_seq_gen", sequenceName = "news_seq", allocationSize
= 1)
    private Long id;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    @org.hibernate.annotations.OnDelete(action =
org.hibernate.annotations.OnDeleteAction.CASCADE)
    private User user;
    @Column(nullable = false, unique = true, length = 64)
    private String title;

```

```

@Column(nullable = false, length = 2048)
private String content;
@CreationTimestamp
private LocalDateTime created;
@UpdateTimestamp
private LocalDateTime modified;
@Builder.Default
@ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
@JoinTable(
    name = "tbl_news_marker",
    joinColumns = @JoinColumn(name = "news_id"),
    inverseJoinColumns = @JoinColumn(name = "marker_id")
)
private Set<Marker> markers = new HashSet<>();
}

```

=====

File: Files from 4/NewsApiApplication.java

=====

```

package org.example.newsapi;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class NewsApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(NewsApiApplication.class, args);
    }
}

```

=====

File: Files from 4/NewsController.java

=====

```

package org.example.newsapi.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;
import org.example.newsapi.service.NewsService;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/v1.0/news")
@RequiredArgsConstructor

```

```

public class NewsController {
    private final NewsService newsService;
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public NewsResponseTo create(@RequestBody @Valid NewsRequestTo request) {
        return newsService.create(request);
    }
    @GetMapping
    public List<NewsResponseTo> getAll(@PageableDefault(size = 50) Pageable pageable) {
        return newsService.findAll(pageable).getContent();
    }
    @GetMapping("/{id}")
    public NewsResponseTo getById(@PathVariable Long id) {
        return newsService.findById(id);
    }
    @PutMapping("/{id}")
    public NewsResponseTo update(@PathVariable Long id, @RequestBody @Valid
NewsRequestTo request) {
        return newsService.update(id, request);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        newsService.delete(id);
    }
}

```

=====
File: Files from 4/NewsMapper.java
=====

```

package org.example.newsapi.mapper;
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;
import org.example.newsapi.entity.Marker;
import org.example.newsapi.entity.News;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;
import java.util.Set;
import java.util.stream.Collectors;
@Mapper(componentModel = "spring", imports = {Collectors.class, Set.class, Marker.class})
public interface NewsMapper {
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "user", ignore = true)
    @Mapping(target = "markers", ignore = true)
    @Mapping(target = "created", ignore = true)

```

```

    @Mapping(target = "modified", ignore = true)
    News toEntity(NewsRequestTo request);
    @Mapping(target = "marker", ignore = true)
    @Mapping(target = "userId", source = "user.id")
    @Mapping(target = "markerIds", expression = "java(news.getMarkers() != null ?
news.getMarkers().stream().map(Marker::getId).collect(Collectors.toSet())) : new
java.util.HashSet<>())")
    NewsResponseTo toDto(News news);
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "user", ignore = true)
    @Mapping(target = "markers", ignore = true)
    @Mapping(target = "created", ignore = true)
    @Mapping(target = "modified", ignore = true)
    void updateEntityFromDto(NewsRequestTo request, @MappingTarget News news);
}

```

=====

File: Files from 4/NewsRepository.java

=====

```

package org.example.newsapi.repository;
import org.example.newsapi.entity.News;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.stereotype.Repository;
@Repository
public interface NewsRepository extends JpaRepository<News, Long>,
JpaSpecificationExecutor<News> {
    boolean existsByTitle(String title);
}

```

=====

File: Files from 4/NewsRequestTo.java

=====

```

package org.example.newsapi.dto.request;
import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;
import java.util.Set;
@Data
public class NewsRequestTo {
    @NotNull
    private Long userId;
    @Size(min = 2, max = 64)
    private String title;
    @Size(min = 4, max = 2048)

```

```

private String content;
private Set<String> markerNames;
public void setMarker(Set<String> markerNames) {
    System.out.println(">>> setMarker called with: " + markerNames);
    this.markerNames = markerNames;
}
public void setMarkers(Set<String> markerNames) {
    System.out.println(">>> setMarkers called with: " + markerNames);
    this.markerNames = markerNames;
}
}

```

=====

File: Files from 4/NewsResponseTo.java

=====

```

package org.example.newsapi.dto.response;
import com.fasterxml.jackson.annotation.JsonAlias;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Data;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;
@Data
public class NewsResponseTo {
    private Long id;
    private Long userId;
    private String title;
    private String content;
    private LocalDateTime created;
    private LocalDateTime modified;
    private Set<Long> markerIds = new HashSet<>();
    public Long getUser() {
        return this.userId;
    }
    public Set<Long> getMarker() {
        return this.markerIds;
    }
}

```

=====

File: Files from 4/NewsService.java

=====

```

package org.example.newsapi.service;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.NewsRequestTo;
import org.example.newsapi.dto.response.NewsResponseTo;

```

```

import org.example.newsapi.entity.Marker;
import org.example.newsapi.entity.News;
import org.example.newsapi.entity.User;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.NewsMapper;
import org.example.newsapi.repository.MarkerRepository;
import org.example.newsapi.repository.NewsRepository;
import org.example.newsapi.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class NewsService {
    private final NewsRepository newsRepository;
    private final UserRepository userRepository;
    private final MarkerRepository markerRepository;
    private final NewsMapper newsMapper;
    @Transactional
    public NewsResponseTo create(NewsRequestTo request) {
        if (request.getUserId() == null || !userRepository.existsById(request.getUserId())) {
            throw new NotFoundException("User not found");
        }
        if (newsRepository.existsByTitle(request.getTitle())) {
            throw new AlreadyExistsException("News title already exists");
        }
        User user = userRepository.getReferenceById(request.getUserId());
        News news = newsMapper.toEntity(request);
        news.setUser(user);
        news.setCreated(LocalDateTime.now());
        news.setModified(LocalDateTime.now());
        if (request.getMarkerNames() != null && !request.getMarkerNames().isEmpty()) {
            Set<Marker> markers = new HashSet<>();
            for (String name : request.getMarkerNames()) {
                Marker marker = markerRepository.findByName(name)
                    .orElseGet(() -> {
                        Marker newMarker = Marker.builder().name(name).build();
                        return markerRepository.save(newMarker);
                    });
                markers.add(marker);
            }
        }
    }
}

```

```

        }
        news.setMarkers(markers);
    }
    News saved = newsRepository.saveAndFlush(news);
    return newsMapper.toDto(saved);
}
public Page<NewsResponseTo> findAll(Pageable pageable) {
    return newsRepository.findAll(pageable).map(newsMapper::toDto);
}
public NewsResponseTo findById(Long id) {
    return newsRepository.findById(id)
        .map(newsMapper::toDto)
        .orElseThrow(() -> new NotFoundException("News not found"));
}
@Transactional
public NewsResponseTo update(Long id, NewsRequestTo request) {
    News news = newsRepository.findById(id)
        .orElseThrow(() -> new NotFoundException("News not found"));
    if (!userRepository.existsById(request.getUserId())) {
        throw new NotFoundException("User not found");
    }
    newsMapper.updateEntityFromDto(request, news);
    news.setUser(userRepository.getReferenceById(request.getUserId()));
    news.setModified(LocalDate.now());
    if (request.getMarkerNames() != null) {
        Set<Marker> markers = new HashSet<>();
        for (String name : request.getMarkerNames()) {
            Marker marker = markerRepository.findByName(name)
                .orElseGet(() -> {
                    Marker newMarker = Marker.builder().name(name).build();
                    return markerRepository.save(newMarker);
                });
            markers.add(marker);
        }
        news.setMarkers(markers);
    } else {
    }
    return newsMapper.toDto(newsRepository.save(news));
}
@Transactional
public void delete(Long id) {
    if (!newsRepository.existsById(id)) {
        throw new NotFoundException("News not found");
    }
    newsRepository.deleteById(id);
}

```

```
}
```

```
=====
File: Files from 4/NotFoundException_1.java
=====
```

```
package org.example.newsapi.exception;
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
```

```
=====
File: Files from 4/NotFoundException.java
=====
```

```
package org.example.discussion.exception;
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}
```

```
=====
File: Files from 4/schema.cql
=====
```

```
CREATE KEYSPACE IF NOT EXISTS distcomp
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE IF NOT EXISTS distcomp.tbl_comment (
    id bigint PRIMARY KEY,
    news_id bigint,
    content text,
    state text
);

CREATE INDEX IF NOT EXISTS idx_comment_news_id ON distcomp.tbl_comment
(news_id);
```

```
=====
File: Files from 4/User.java
=====
```

```
package org.example.newsapi.entity;
import jakarta.persistence.*;
import lombok.*;
import java.util.List;
@Entity
```

```

@Table(name = "tbl_user")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "user_seq_gen")
    @SequenceGenerator(name = "user_seq_gen", sequenceName = "user_seq", allocationSize =
1)
    private Long id;
    @Column(nullable = false, unique = true, length = 64)
    private String login;
    @Column(nullable = false, length = 128)
    private String password;
    @Column(length = 64)
    private String firstname;
    @Column(length = 64)
    private String lastname;
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @ToString.Exclude
    private List<News> news;
}

```

=====

File: Files from 4/UserController.java

=====

```

package org.example.newsapi.controller;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.UserRequestTo;
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.service.UserService;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/v1.0/users")
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserResponseTo create(@RequestBody @Valid UserRequestTo request) {

```

```

        return userService.create(request);
    }
    @GetMapping
    public List<UserResponseTo> getAll(@PageableDefault(size = 50) Pageable pageable) {
        return userService.findAll(pageable).getContent();
    }
    @GetMapping("/{id}")
    public UserResponseTo getById(@PathVariable Long id) {
        return userService.findById(id);
    }
    @PutMapping("/{id}")
    public UserResponseTo update(@PathVariable Long id, @RequestBody @Valid
UserRequestTo request) {
        return userService.update(id, request);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        userService.delete(id);
    }
}

```

=====

File: Files from 4/UserMapper.java

=====

```

package org.example.newsapi.mapper;
import org.example.newsapi.dto.request.UserRequestTo;
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.entity.User;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;
@Mapper(componentModel = "spring")
public interface UserMapper {
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true)
    User toEntity(UserRequestTo request);
    UserResponseTo toDto(User user);
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "news", ignore = true)
    void updateEntityFromDto(UserRequestTo request, @MappingTarget User user);
}

```

=====

File: Files from 4/UserRepository.java

=====

```

package org.example.newsapi.repository;
import org.example.newsapi.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    boolean existsByLogin(String login);
}

```

=====

File: Files from 4/UserRequestTo.java

=====

```

package org.example.newsapi.dto.request;
import jakarta.validation.constraints.Size;
import lombok.Data;
@Data
public class UserRequestTo {
    @Size(min = 2, max = 64)
    private String login;
    @Size(min = 8, max = 128)
    private String password;
    @Size(min = 2, max = 64)
    private String firstname;
    @Size(min = 2, max = 64)
    private String lastname;
}

```

=====

File: Files from 4/UserResponseTo.java

=====

```

package org.example.newsapi.dto.response;
import com.fasterxml.jackson.annotation.JsonTypeName;
import com.fasterxml.jackson.annotation.JsonTypeInfo;
import com.fasterxml.jackson.annotation.JsonRootName;
import lombok.Data;
@Data
@JsonRootName("user")
public class UserResponseTo {
    private Long id;
    private String login;
    private String firstname;
    private String lastname;
}

```

=====

File: Files from 4/UserService.java

=====

```
package org.example.newsapi.service;
import lombok.RequiredArgsConstructor;
import org.example.newsapi.dto.request.UserRequestTo;
import org.example.newsapi.dto.response.UserResponseTo;
import org.example.newsapi.entity.User;
import org.example.newsapi.exception.AlreadyExistsException;
import org.example.newsapi.exception.NotFoundException;
import org.example.newsapi.mapper.UserMapper;
import org.example.newsapi.repository.UserRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class UserService {
    private final UserRepository userRepository;
    private final UserMapper userMapper;
    @Transactional
    public UserResponseTo create(UserRequestTo request) {
        if (userRepository.existsByLogin(request.getLogin())) {
            throw new AlreadyExistsException("Login already exists");
        }
        User user = userMapper.toEntity(request);
        return userMapper.toDto(userRepository.save(user));
    }
    public Page<UserResponseTo> findAll(Pageable pageable) {
        return userRepository.findAll(pageable)
            .map(userMapper::toDto);
    }
    public UserResponseTo findById(Long id) {
        return userRepository.findById(id)
            .map(userMapper::toDto)
            .orElseThrow(() -> new NotFoundException("User not found with id: " + id));
    }
    @Transactional
    public UserResponseTo update(Long id, UserRequestTo request) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("User not found with id: " + id));
        userMapper.updateEntityFromDto(request, user);
        return userMapper.toDto(userRepository.save(user));
    }
    @Transactional
    public void delete(Long id) {
```

```

        if (!userRepository.existsById(id)) {
            throw new NotFoundException("User not found with id: " + id);
        }
        userRepository.deleteById(id);
    }
}

```

=====
File: Files from 4/v1.0.0-create-tables.xml
=====

```

<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
    xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.24.xsd">
    <changeSet id="1-create-sequences" author="brazhalovich">
        <createSequence sequenceName="user_seq" incrementBy="1" startValue="1"/>
        <createSequence sequenceName="news_seq" incrementBy="1" startValue="1"/>
        <createSequence sequenceName="marker_seq" incrementBy="1" startValue="1"/>
    </changeSet>
    <changeSet id="2-create-tbl-user" author="brazhalovich">
        <createTable tableName="tbl_user">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('user_seq')"><constraints primaryKey="true"
nullable="false"/></column>
            <column name="login" type="VARCHAR(64)"><constraints nullable="false"
unique="true"/></column>
            <column name="password" type="VARCHAR(128)"><constraints
nullable="false"/></column>
            <column name="firstname" type="VARCHAR(64)"><column name="lastname"
type="VARCHAR(64)">
        </createTable>
    </changeSet>
    <changeSet id="3-create-tbl-news" author="brazhalovich">
        <createTable tableName="tbl_news">
            <column name="id" type="BIGINT"
defaultValueComputed="nextval('news_seq')"><constraints primaryKey="true"
nullable="false"/></column>
            <column name="user_id" type="BIGINT"><constraints nullable="false"/></column>
            <column name="title" type="VARCHAR(64)"><constraints nullable="false"
unique="true"/></column>
            <column name="content" type="VARCHAR(2048)"><constraints
nullable="false"/></column>
            <column name="created" type="TIMESTAMP"/><column name="modified"
type="TIMESTAMP"/>

```

```

    </createTable>
    <addForeignKeyConstraint baseTableName="tbl_news" baseColumnNames="user_id"
constraintName="fk_news_user" referencedTableName="tbl_user"
referencedColumnNames="id" onDelete="CASCADE"/>
  </changeSet>
  <changeSet id="4-create-tbl-marker" author="brazhalovich">
    <createTable tableName="tbl_marker">
      <column name="id" type="BIGINT"
defaultValueComputed="nextval('marker_seq')"><constraints primaryKey="true"
nullable="false"/></column>
      <column name="name" type="VARCHAR(32)"><constraints nullable="false"
unique="true"/></column>
    </createTable>
  </changeSet>
  <changeSet id="6-create-tbl-news-marker" author="brazhalovich">
    <createTable tableName="tbl_news_marker">
      <column name="news_id" type="BIGINT"><constraints nullable="false"/></column>
      <column name="marker_id" type="BIGINT"><constraints nullable="false"/></column>
    </createTable>
    <addPrimaryKey tableName="tbl_news_marker" columnNames="news_id, marker_id"/>
    <addForeignKeyConstraint baseTableName="tbl_news_marker"
baseColumnNames="news_id" constraintName="fk_nm_news"
referencedTableName="tbl_news" referencedColumnNames="id" onDelete="CASCADE"/>
    <addForeignKeyConstraint baseTableName="tbl_news_marker"
baseColumnNames="marker_id" constraintName="fk_nm_marker"
referencedTableName="tbl_marker" referencedColumnNames="id" onDelete="CASCADE"/>
  </changeSet>
  <changeSet id="7-insert-initial-user" author="brazhalovich">
    <insert tableName="tbl_user">
      <column name="id" valueNumeric="1"/>
      <column name="login" value="sashabrazhalovich2005@gmail.com"/>
      <column name="password" value="temp_password"/>
      <column name="firstname" value="Александр"/>
      <column name="lastname" value="Бражалович"/>
    </insert>
    <sql>SELECT setval('user_seq', 1);</sql>
  </changeSet>
  <changeSet id="8-insert-initial-markers" author="brazhalovich">
    <insert tableName="tbl_marker">
      <column name="id" valueNumeric="1"/>
      <column name="name" value="red2"/>
    </insert>
    <insert tableName="tbl_marker">
      <column name="id" valueNumeric="2"/>
      <column name="name" value="green2"/>
    </insert>
  </changeSet>

```

```
<insert tableName="tbl_marker">
  <column name="id" valueNumeric="3"/>
  <column name="name" value="blue2"/>
</insert>
<sql>SELECT setval('marker_seq', 3);</sql>
</changeSet>
</databaseChangeLog>
```

=====

File: Files from 4/WebClientConfig.java

=====

```
package org.example.newsapi.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;
@Configuration
public class WebClientConfig {
    @Bean
    public WebClient discussionWebClient() {
        return WebClient.builder()
            .baseUrl("http://localhost:24130/api/v1.0/comments")
            .build();
    }
}
```

=====

End of Codebase

=====